写在前面

本实验手册致力于帮助各位同学完成第四次课程的实战项目-《基于 BKVision 图表平台实现游戏业务主机管理》，在此之前，请各位同学确认自己已经完成了第二次和第三次课程的实战项目，并有一个可以访问的部署好了的 SaaS 应用，确保已经实现了【基于 CMDB 配置平台进行游戏业务主机查询】和【基于 JOB 作业平台进行游戏业务文件备份】。

本次实战项目将基于此前实现的接口，通过设计自定义 Django 中间件，实现用户行为埋点记录，并存储用户行为数据到数据库中，如：登录行为、主机查询行为、文件查询行为、文件备份行为等，并通过【BKVision 图表平台】进行数据可视化呈现，将设计好的数据仪表盘嵌入到我们的前端网站中，以实现用户画像。

- BKVision 图表平台：蓝鲸图表平台 BkVision
- Django 中间件解析：Django 中间件原理及示例
- 示例 DEMO：基于 BKVision 图表平台实现游戏业务主机管理

做前必读

实验四是在实验三的基础上增加了一些功能，因此可以使用自己已完成的实验三的代码进行迭代开发。开发后端模块时，也可以使用提供的后端代码包进行二次开发，不要使用从开发者中心下载的模板代码，避免环境不一致。

Step0-需求分析：理清实验要求

实验要求：BKVision - 实验要求

核心需求总结如下：

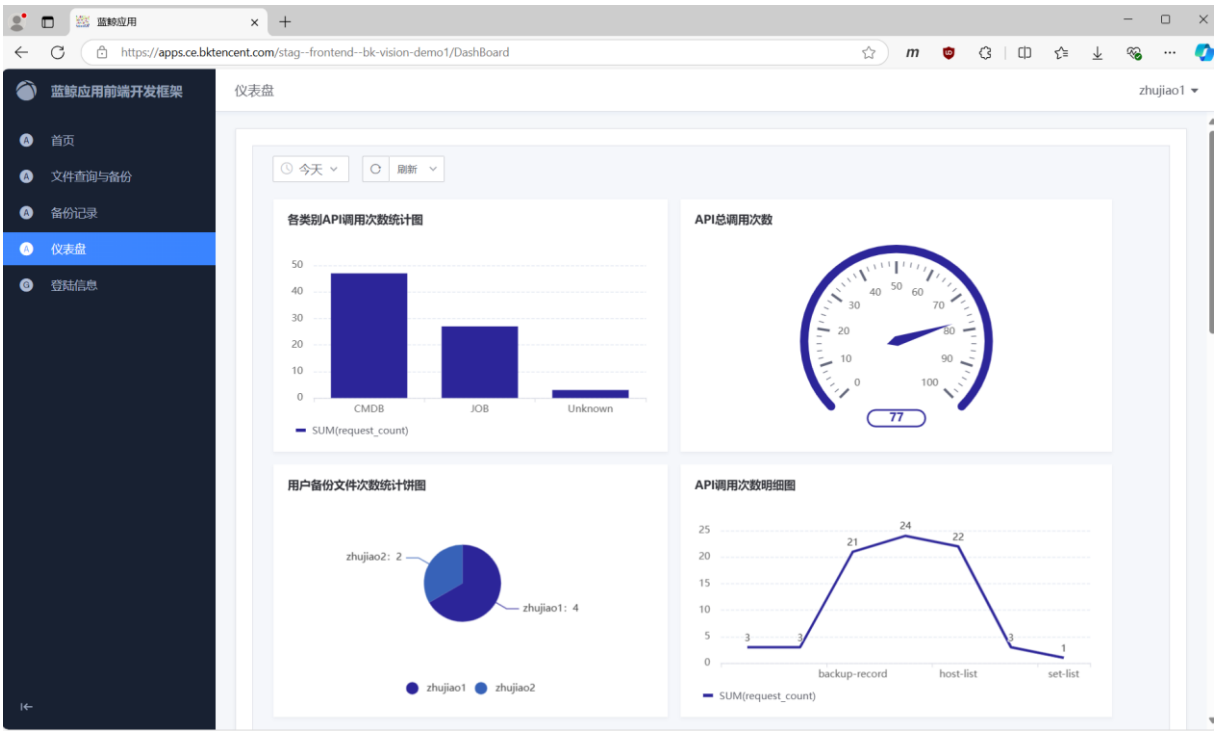
- 在此前二三次课程实战项目的基础上，设计 Django 中间件与数据存储 Model，实现用户行为采集，并存储记录到数据库中
- 在 BKVision 平台接入自己的 SaaS 数据库，配置数据源
- 在 BKVision 平台，使用自己的数据源，实现用户画像仪表盘
- 借助 iFrame/BKVision-SDK，将仪表盘嵌入到自己的 SaaS 应用的前端界面中

本手册将带领大家分别实现四个仪表盘：

- API 调用类别统计柱状图-统计一期项目 CMDB 和二期项目 JOB 的接口调用次数
- API 总调用次数仪表盘-统计所有 API 的总调用次数
- 用户备份文件次数统计饼图-统计各个用户的备份次数
- API 调用次数明细折线图-统计各个 API 的调用次数明细（如查询业务列表、查询集群列表等）

最终实现的效果如下：[BKVision 仪表板示例]

嵌入效果如下：



Step1-设计实现用户行为数据存储 Model

在实验手册案例中，我们将实现数据存储 Model，存储两类数据：

- API 请求次数记录表，以请求的 API 类别（如 CMDB、JOB）和请求的 API 名称（如 biz-list、set-list、search-file）为索引

这里在 home_application/models.py 中实现，实际编码时，可以根据自己的项目目录结构进行修改。

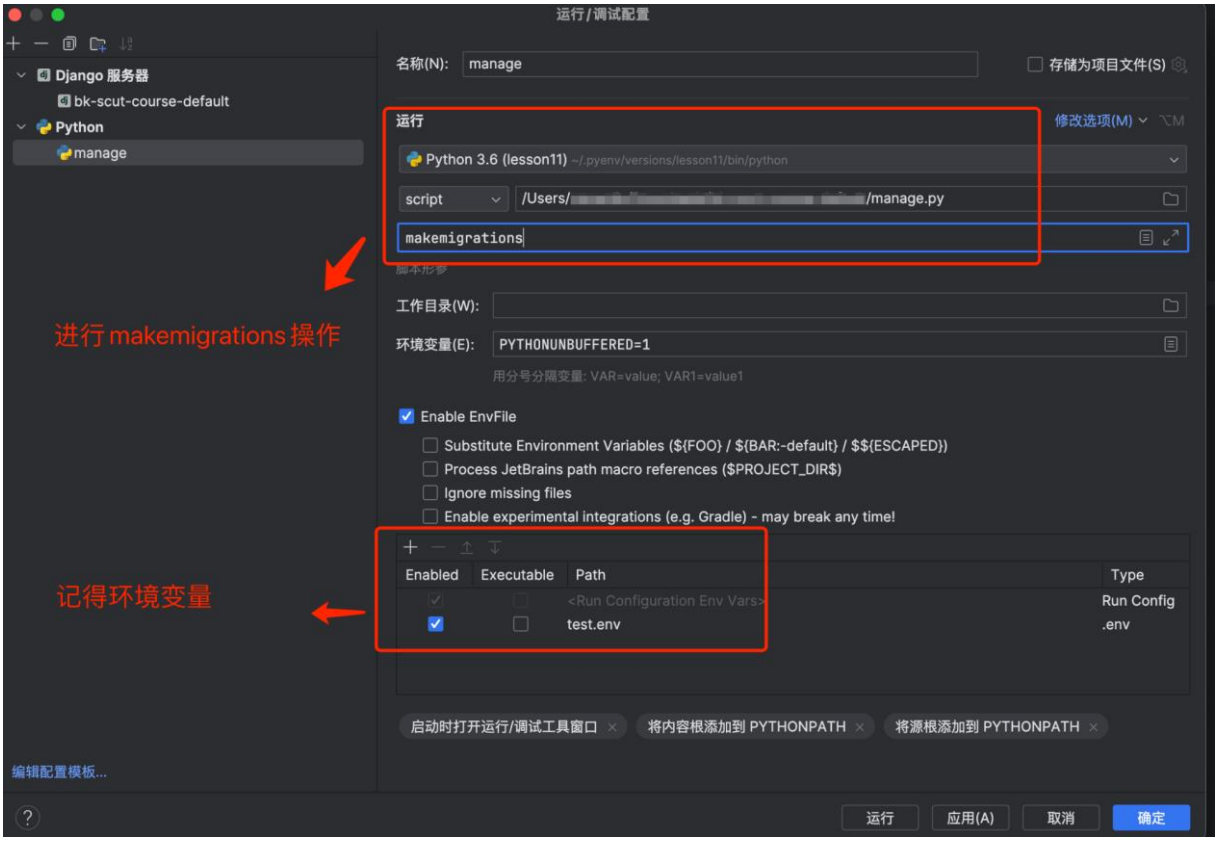
实现 API 请求次数记录表数据 Model

在 home_application/models.py 中实现，这里通过设置 api_category 和 api_name 为联合索引，确保每个 API 只存在一条记录，后续增加 request_count 的值即可。

```
class ApiRequestCount(models.Model):
    """
    API 请求次数记录模型，用于运营分析
    """

    api_category = models.CharField(verbose_name="API 类别",
max_length=255)
    api_name = models.CharField(verbose_name="API 名称",
max_length=255)
    request_count = models.IntegerField(verbose_name="请求次数",
default=0)

    class Meta:
        unique_together = ("api_category", "api_name")  # 联合唯一
索引
        verbose_name = "API 请求次数"
        verbose_name_plural = "API 请求次数"

    def __str__(self):
        return f"{self.api_category}-{self.api_name}"
```

Step2-执行数据库迁移，makemigrations&migrate

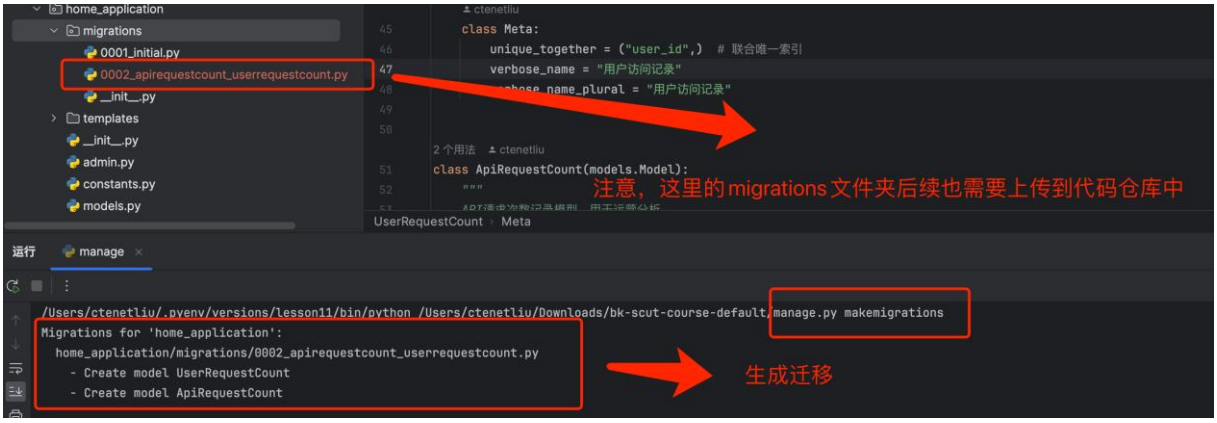
和此前实验三的项目一样，实现好数据 Model 后，我们需要将其迁移到数据库中，分别执行 python manage.py makemigrations 和 python manage.py migrate



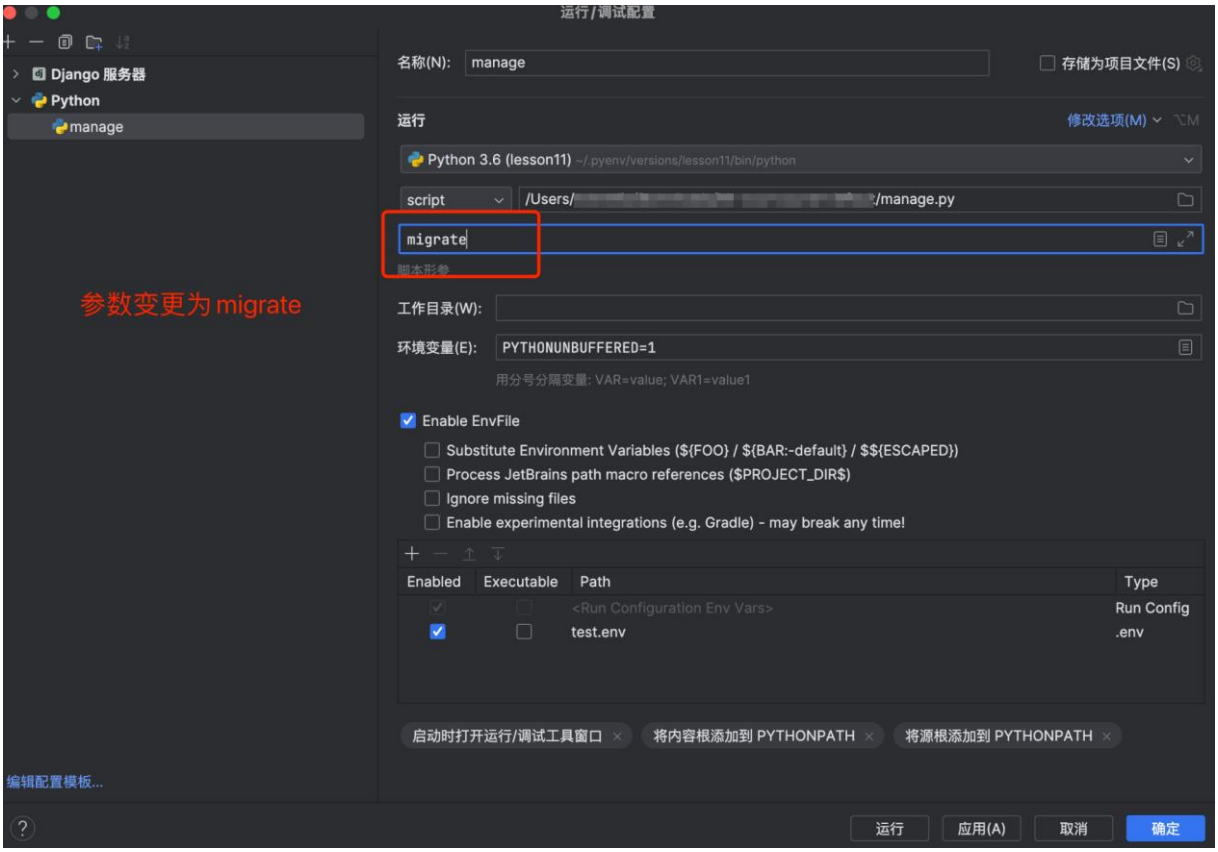
makemigrations

执行 makemigrations 操作，输出如下

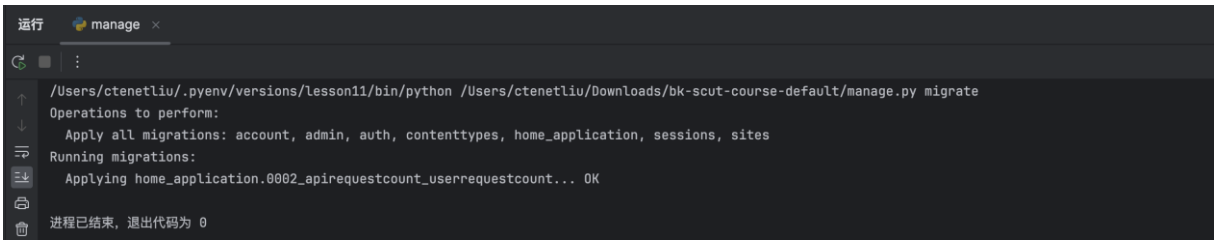
注意：这里生成的 migrations 文件夹和其中的文件，后续也需要上传到代码仓库中



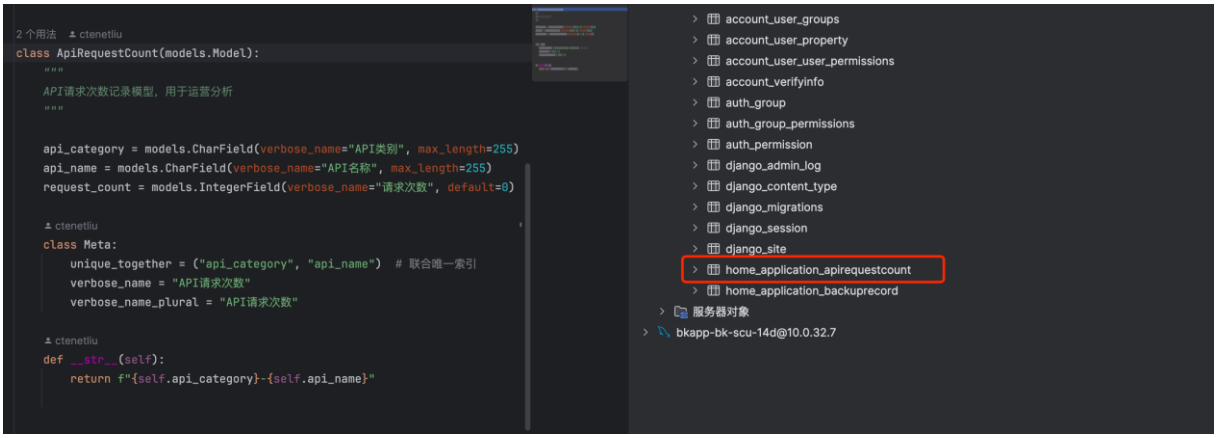
migrate



执行 migrate 操作，输出如下：



成功后，可以看到数据库中新增了一张数据表，分别对应此前新增的数据 Model：



Step3-设计自定义中间件，实现用户行为埋点记录

Django 中间件简单介绍

这里简单介绍一下 Django 中间件，详细可参考教程：[Django 中间件原理及示例]

- 中间件是一个轻量级、底层级别的插件系统，可以介入 Django 生命周期中的请求和响应的处理过程，控制 Django 程序的输入和输出
- 使用：在 Django 配置文件中的 MIDDLEWARE 中注册即可
- CSRF 中间件：'django.middleware.csrf.CsrfViewMiddleware'
- Django 中的中间件最多可以定义五个方法：

```
process_request(self,request)
process_view(self, request, view_func, view_args, view_kwargs)
process_template_response(self,request,response)
process_exception(self, request, exception)
process_response(self, request, response)
```

Django 中的中间件的执行顺序：

1. 请求进入到 Django 后，Django 接收到第一个请求会调用中间件的初始化方法 __init__，处理请求之前会按中间件的注册顺序执行每个中间件中的 process_request 方法。如果所有的中间件的 process_request 方法 return None，则进入路由映射，进行 url 匹配，通过规则确认请求由哪个视图处理,如果是 return HTTPResponse，则调用 process_response 返回到客户端

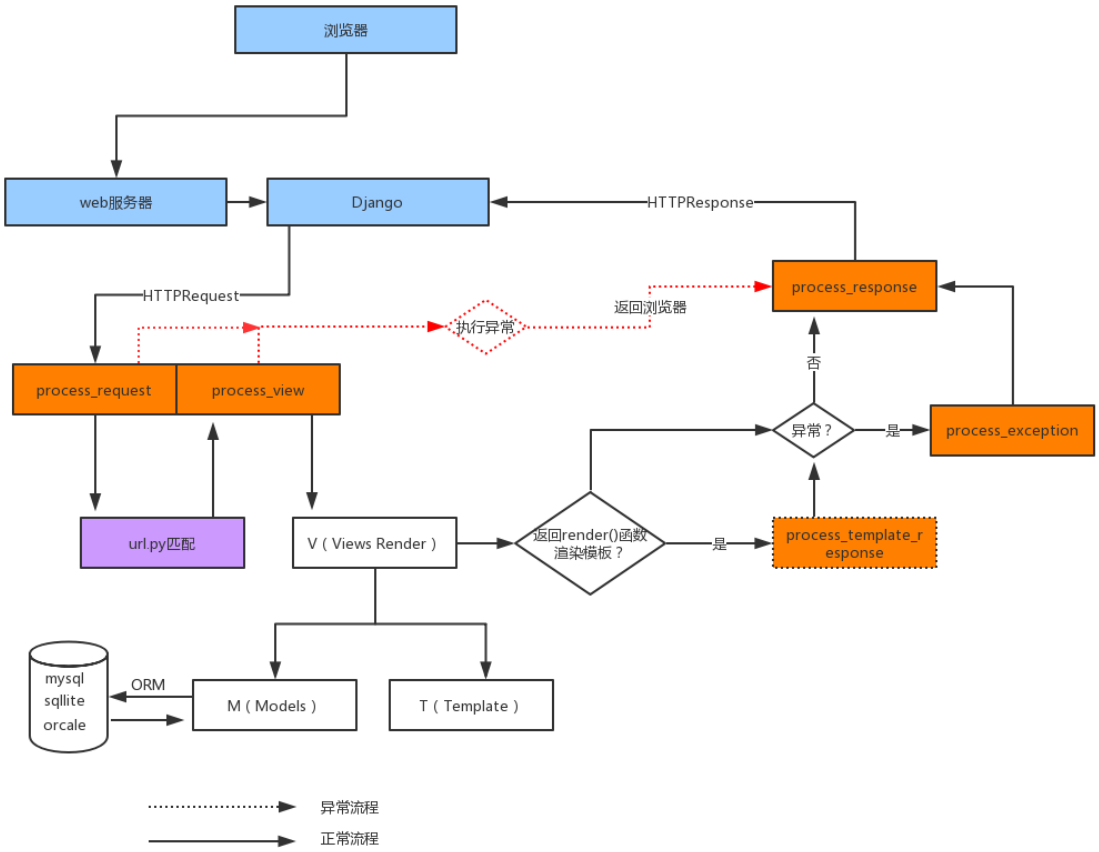
2. 依次按顺序执行中间件中的 process_view 方法

   如果某个中间件的 process_view 方法没有 return HttpResponse，则根据第 1 步中匹配到的 URL 执行对应的视图函数或视图类（process_view）在 views.py 执行之前如果某个中间件的 process_view 方法中返回了 return HttpResponse，则后面的视图函数或视图类不会执行,程序会执行 process_response 返回客户端

3. 视图函数或者视图类接收 Request，通过模型 Model 与数据库交互，获取数据交给模板引擎进行渲染，如果视图函数或视图类中使用 render()方法来向客户端返回数据，则会触发中间件中的 process_template_response 方法,response 函数都是通过注册的逆序进行调用，必须返回 response 才能继续转发，否则程序排除异常

4. **随后视图函数或视图类执行计算渲染，如果没有任何异常，会按照中间件的注册顺序逆序执行中间件中的 process_response 方法 如果中间件中定义了 return response，程序会正常执行，把视图函数或视图类的执行结果返回给客户端，否则程序会抛出异常**

5. 在第三步，程序在视图函数或视图类的正常执行过程中 如果出现异常，则会执行按顺序执行中间件中的 process_exception 方法，该方法也是逆序执行的，如果某个中间件的 process_exception 方法中定义了 return 语句，则终端传递中间件中的 process_exception 函数，转发给 process_response 处理后返回给客户端



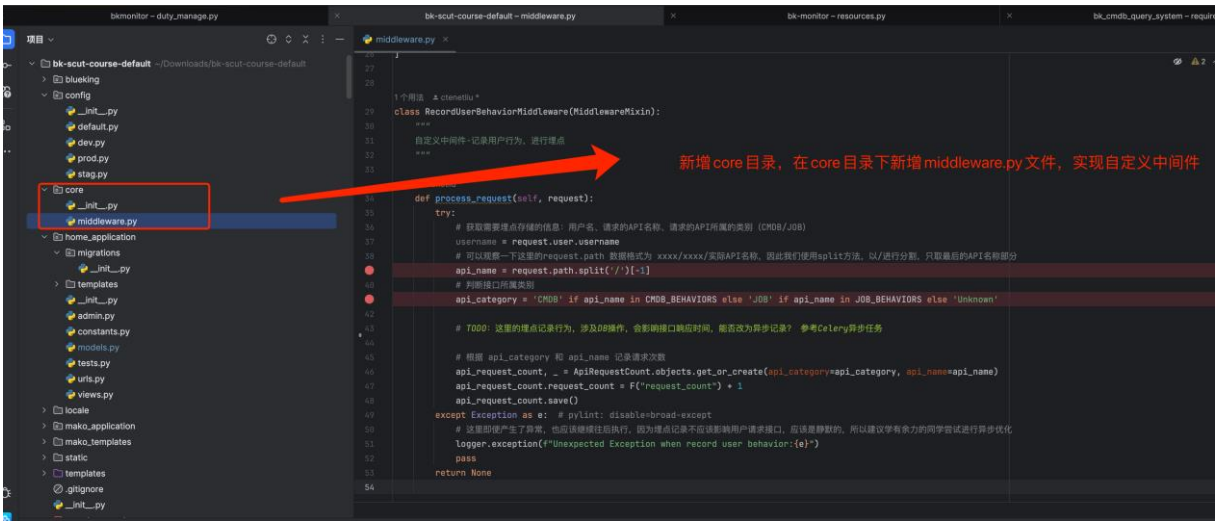
core/middleware.py 完整代码

在仔细讲解之前，这里先给出 core/middleware.py 的完整实现：

```
# -*- coding: utf-8 -*-

from django.utils.deprecation import MiddlewareMixin
import logging
from django.db.models import F


from home_application.models import ApiRequestCount

logger = logging.getLogger(__name__)

# 这里的 CMDB 和 JOB 对应的名称应该同你的 URL 定义的前缀，详见
/home_application/urls.py

CMDB_BEHAVIORS = [
    'biz-list',
    'set-list',
    'module-list',
    'host-list',
    'host-detail'
]

JOB_BEHAVIORS = [
    'search-file',
    'backup-file',
    'backup-record'
]


class RecordUserBehaviorMiddleware(MiddlewareMixin):
    """
    自定义中间件-记录用户行为，进行埋点
    """

    def process_request(self, request):
        try:
            # 获取需要埋点存储的信息：用户名、请求的 API 名称、请求的 API 所
属的类别（CMDB/JOB）
            username = request.user.username
            # 可以观察一下这里的 request.path 数据格式为 xxxx/xxxx/实际
API 名称，因此我们使用 split 方法，以/进行分割，只取最后的 API 名称部分
            api_name = request.path.split('/')[-1]
```

实现自定义中间件-RecordUserBehaviorMiddleware-埋点采集并记录用户行为

我们在项目根目录下新建文件夹 core，在 core 目录下，新增 middleware.py 文件，目录结构如下：

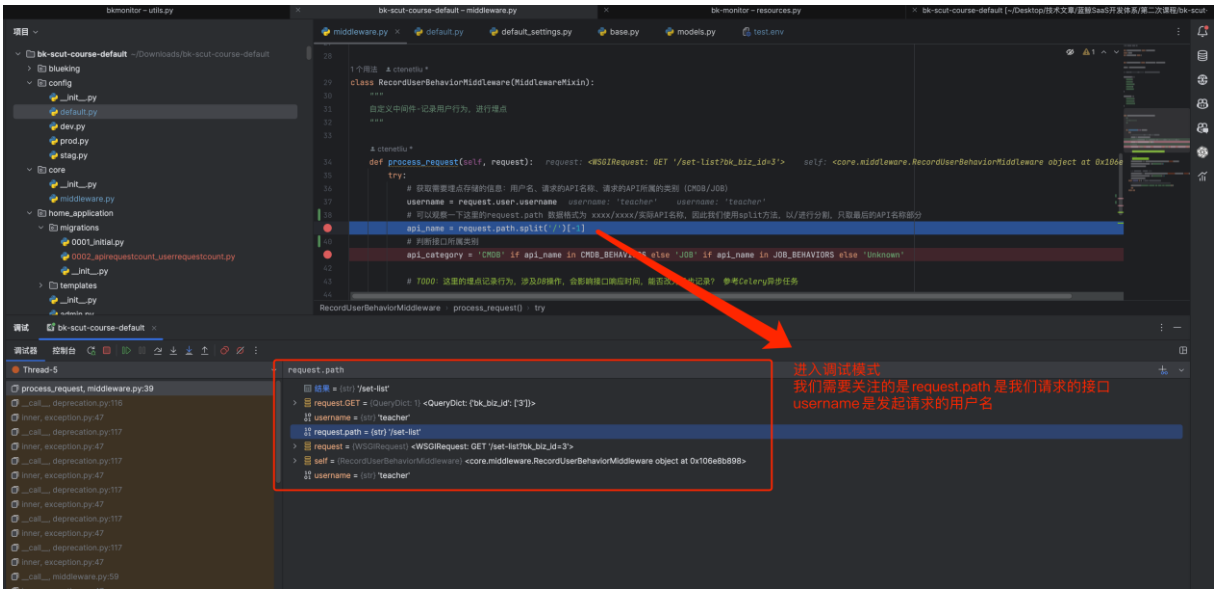


我们的中间件代码如下：

```
class RecordUserBehaviorMiddleware(MiddlewareMixin):
    """
    自定义中间件-记录用户行为，进行埋点
    """

    def process_request(self, request):
        try:
            # 获取需要埋点存储的信息：用户名、请求的 API 名称、请求的 API 所属的类别（CMDB/JOB）
            username = request.user.username
            # 可以观察一下这里的 request.path 数据格式为 xxxx/xxxx/实际 API 名称，因此我们使用 split 方法，以/进行分割，只取最后的 API 名称部分
            api_name = request.path.split('/')[-1]
            # 判断接口所属类别
            api_category = 'CMDB' if api_name in CMDB_BEHAVIORS else 'JOB' if api_name in JOB_BEHAVIORS else 'Unknown'

            # TODO：这里的埋点记录行为，涉及 DB 操作，会影响接口响应时间，能否改为异步记录？ 参考 Celery 异步任务

            # 根据 api_category 和 api_name 记录请求次数
            api_request_count, _ = ApiRequestCount.objects.get_or_create(api_category=api_category, api_name=api_name)
            api_request_count.request_count = F("request_count") + 1
            api_request_count.save()
        except Exception as e:  # pylint: disable=broad-except
            # 这里即使产生了异常，也应该继续往后执行，因为埋点记录不应该影响用户请求接口，应该是静默的，所以建议学有余力的同学尝试进行异步优化
            logger.exception(f"Unexpected Exception when record user behavior:{e}")
            pass
        return None
```

- 这里的核心流程概述如下：
- 通过重写 process_request 方法，实现在用户请求时，拦截并埋点记录

- 从 request.user.username 中获取到用户名
  - 这里需要注意中间件的顺序，如果自定义中间件放在较前的位置的话，会拿不到用户名，因为用户名的处理是在蓝鲸开发框架内置的 blueapps.middleware.request_provider.RequestProvider 中实现的，所以我们的自定义中间件的执行顺序必须在其后
- 从 request.path 中获取到请求的接口名称，并进行 split 分割处理，只取最后的接口名称部分，忽略此前的前缀



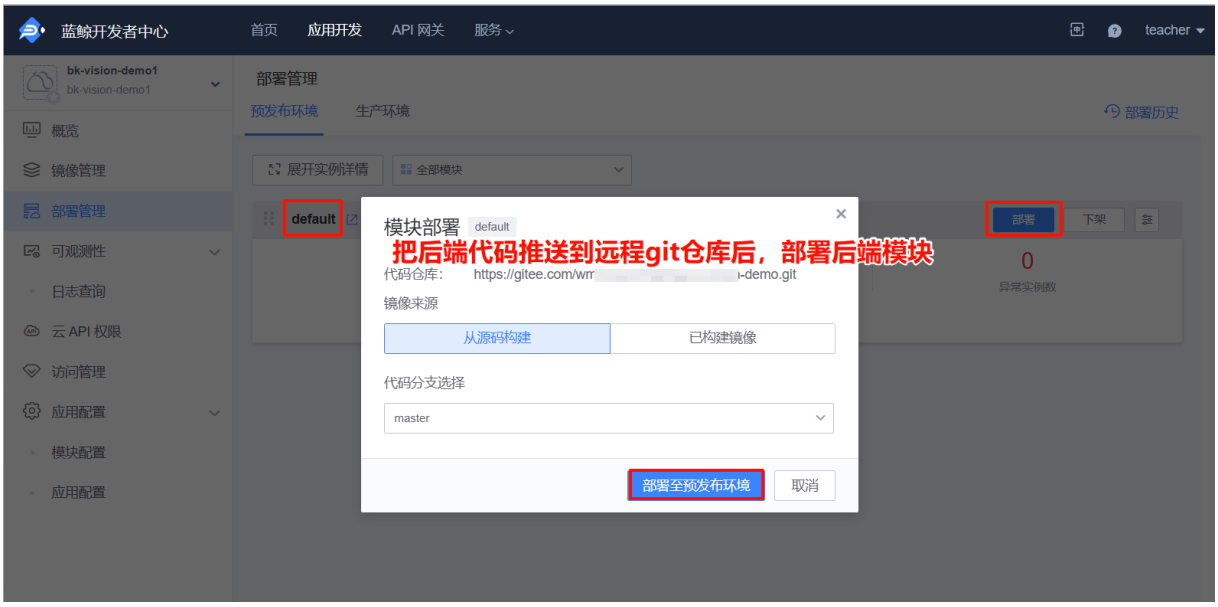
- 根据映射表关系，判断当前接口所属的类别：CMDB/JOB
- 存储行为数据到数据库中，使用 get_or_create 进行，因为每一个 API 只需要一个记录，我们需要做的是递增其 count 值

添加自定义中间件到 config 中

在实现中间件之后，我们还需要将其添加到 config/default.py 中的对应 MIDDLEWARE 中才能生效，具体添加如下：

Step4-推送后端代码并部署上线

和此前两次课程的部署阶段类似，这里我们把后端代码推送到远端仓库，并部署上线



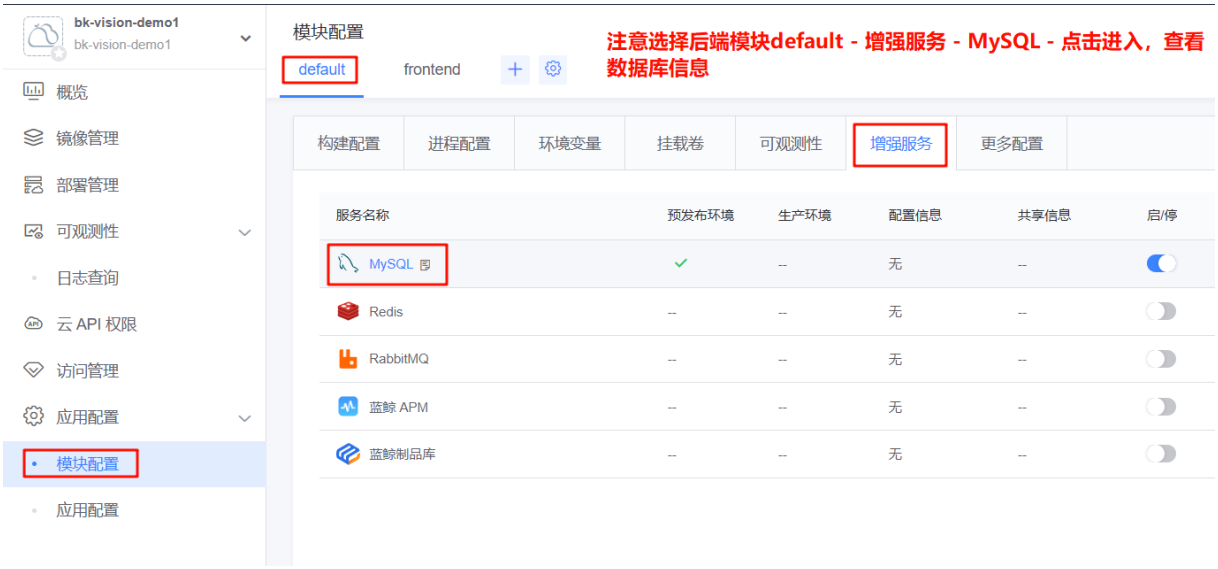
Step5-在 BKVision 平台添加并配置数据源

在完成此前的步骤，设计实现好用户行为埋点记录中间件和数据存储 Model 并将后端代码推送并部署上线后，我们开始着手在 BKVision 平台配置自己的 SaaS 应用数据源，来实现运营仪表盘

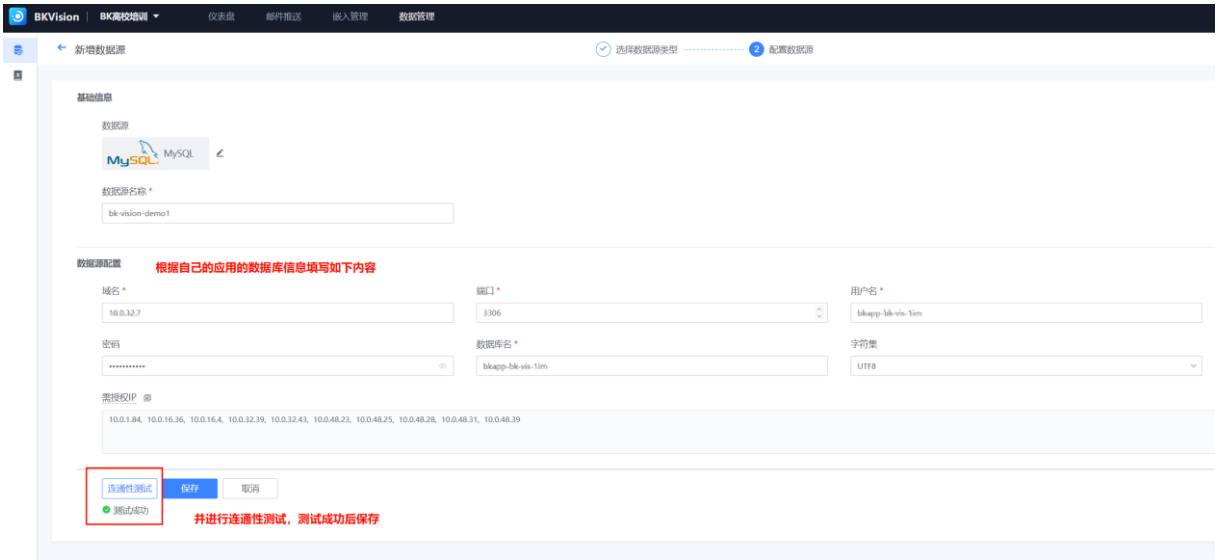
1. 访问 BKVision 平台，选择【BK 高校培训】空间，进入数据管理页面，添加配置自己的 SaaS 应用的数据源

2. 选择【MySQL】数据源

3. 从开发者中心，获取自己的 SaaS 应用的数据库连接信息



4. 将数据库配置信息回填到 BKVision 中，并测试能否成功联通



5. 添加数据源成功：

Step6-创建并配置仪表盘

在 BKVision 平台添加好数据源后，我们开始创建自己的仪表盘

1. 在此前的【BK 高校培训】空间下，新增仪表盘

2. 编辑仪表盘->选择左侧的样式，拖拽到右边->选择此前配置好的数据源，选择需要的数据表

配置实现 API 调用类别柱状统计图

- 这里以实现各类别 API 调用次数统计图来讲解一下简单的图表配置：

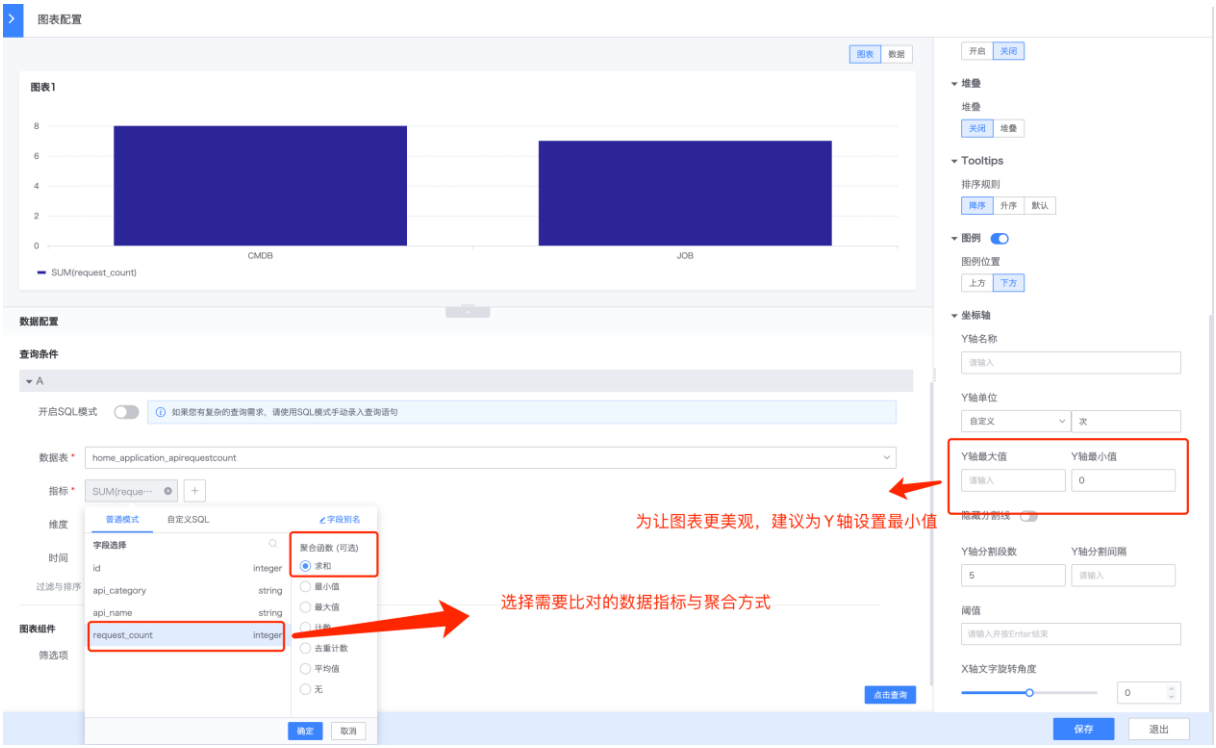
- 在仪表盘->新增->编辑仪表盘，拖拽左侧的柱状图样式，并选择自己的数据源和对应的指标与维度

- 指标：用来衡量某个事物的数量或质量的标准，通常是一个数值或一个比率。

- 维度：用来描述指标的特征或属性的，通常是一个分类变量，对指标进行分类或分组，以便更好地理解和分析数据

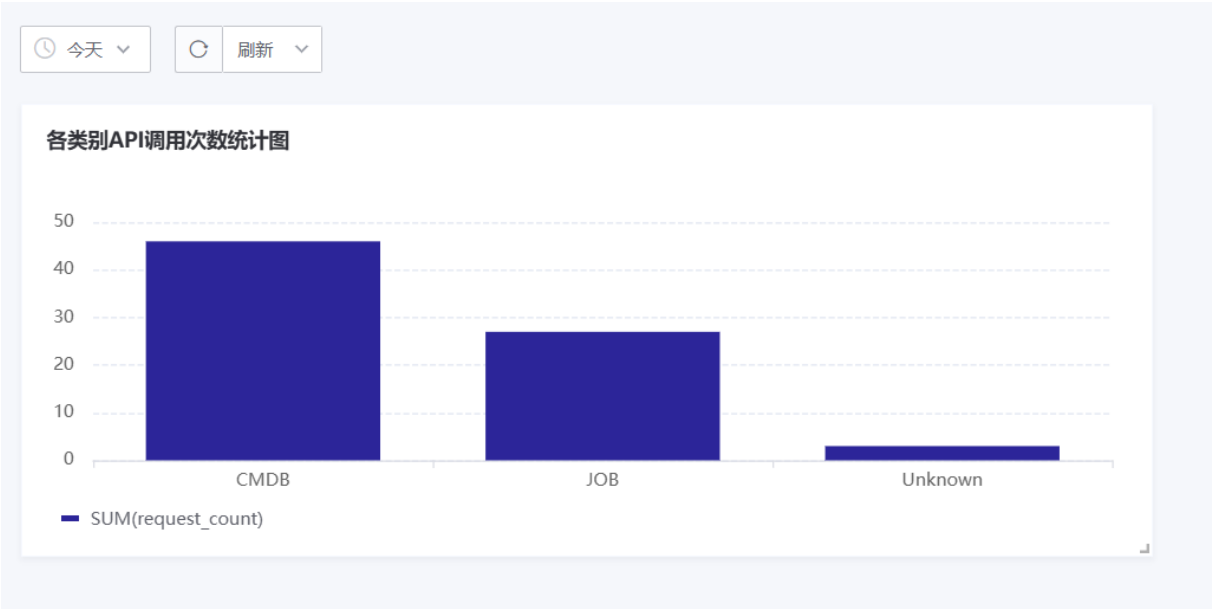
- 直观的讲，维度是我们图表的 X 轴，指标是我们的 Y 轴

- 首先，选择我们的数据表 home_application_apirequestcount，然后选择指标，这里选择的是 request_count（请求次数），聚合方式选择求和

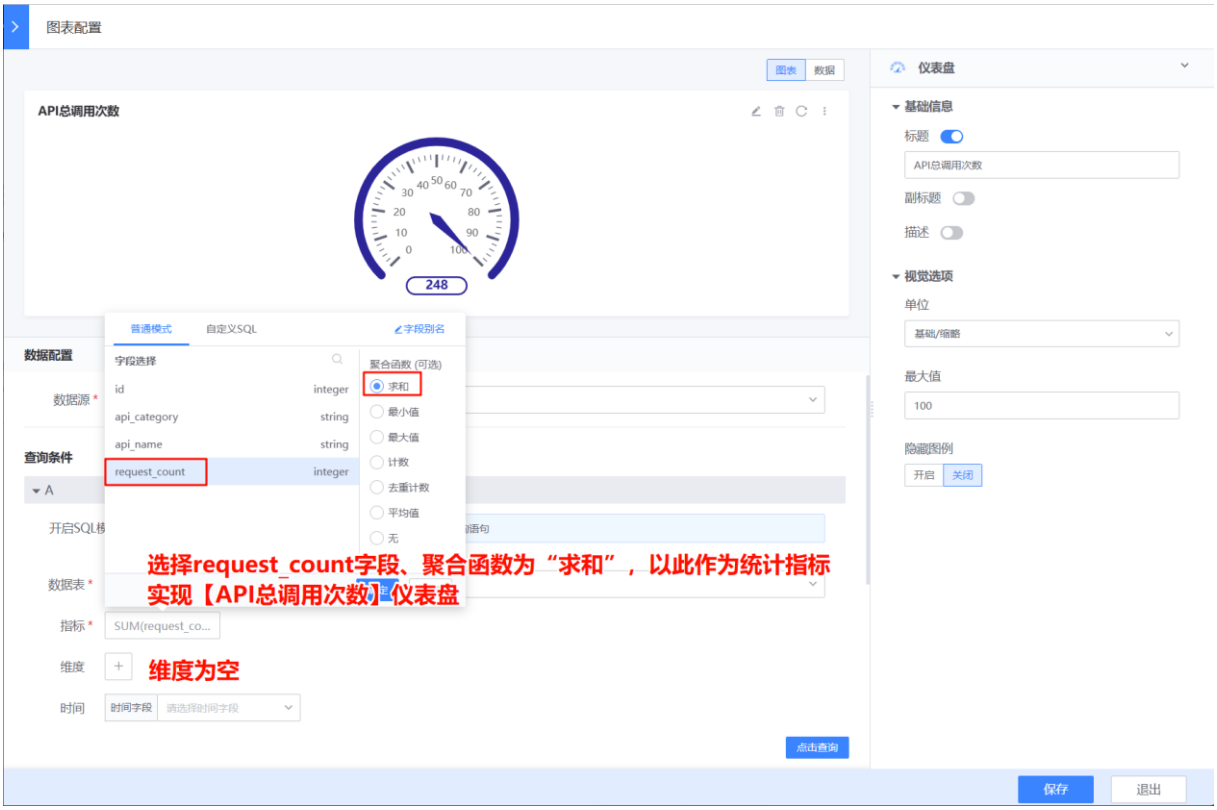


- 然后，选择我们衡量的维度，这里的维度选择 api_category，以 API 类别（CMDB/JOB/Unknown）来统计调用次数：

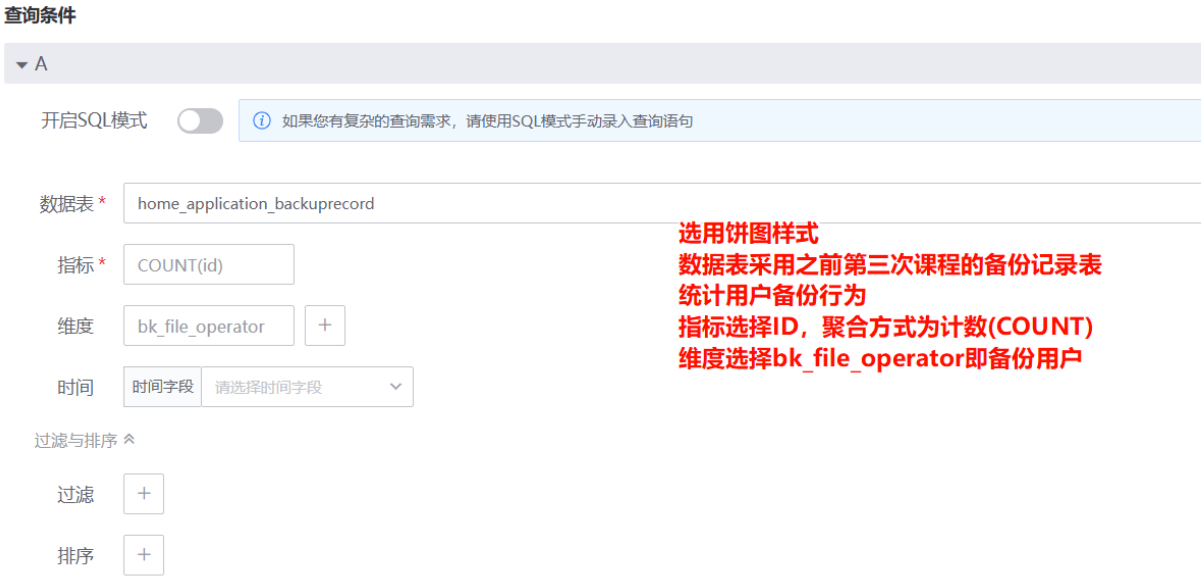
- 最后实现的效果图如下：

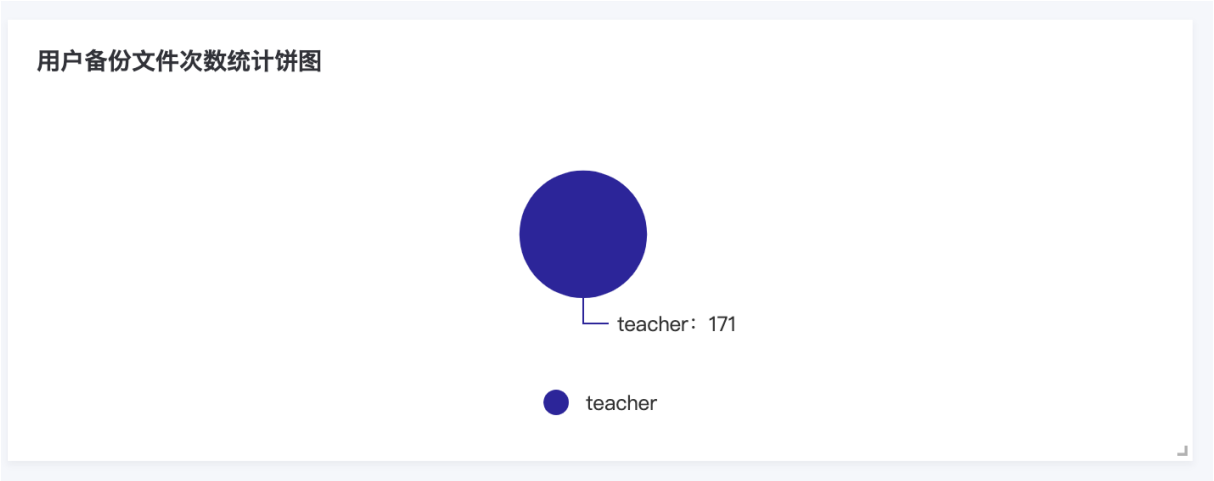


配置实现 API 总调用次数仪表盘

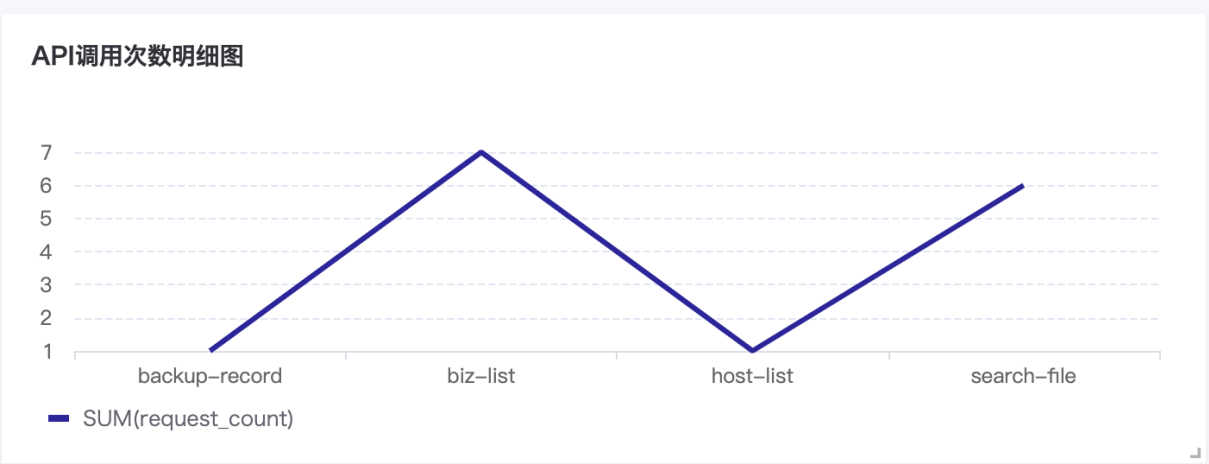


配置实现用户备份文件次数统计饼图

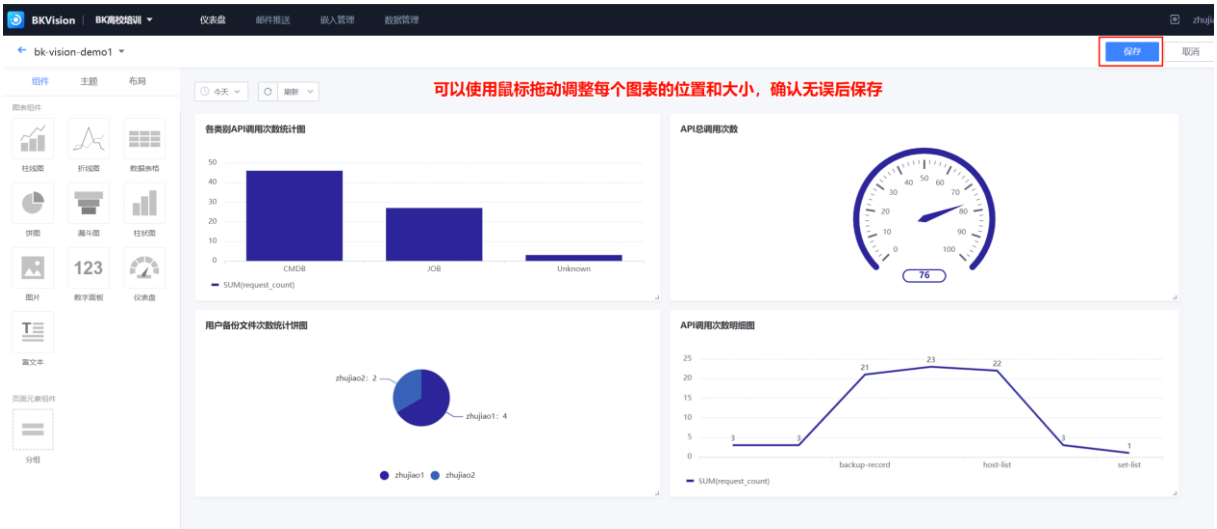




配置实现 API 调用次数明细图



最终的 4 个图表在仪表板设计中的样式如下，可以自行拖动调整每个图表的大小和位置，确认无误后保存。

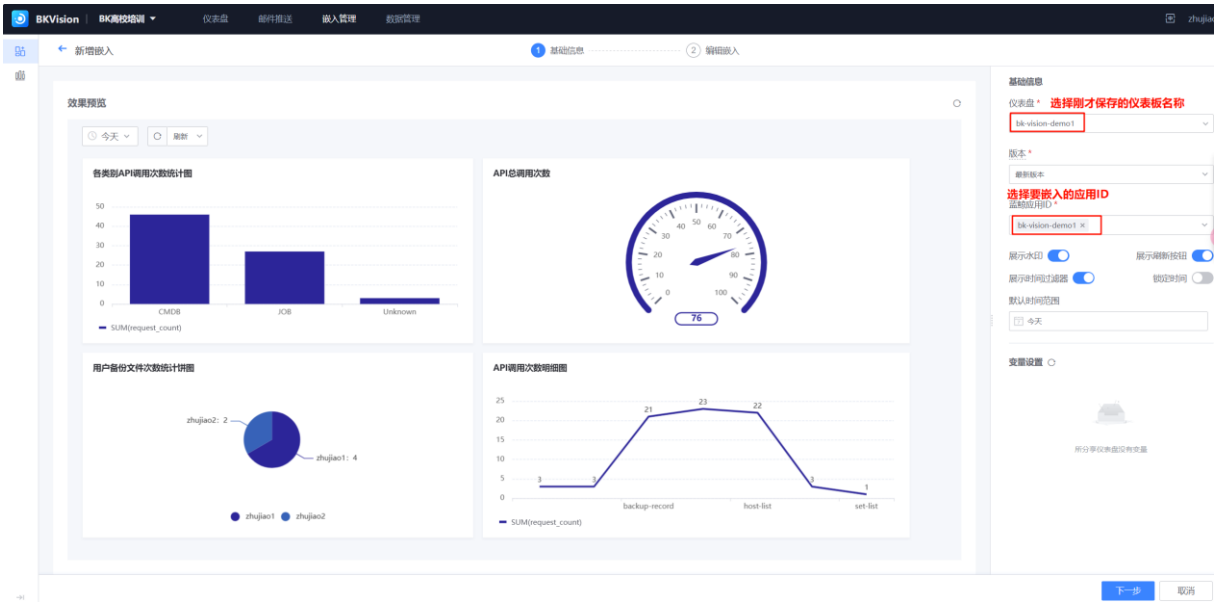


Step7-将仪表盘嵌入到自己的前端应用（以 iFrame 方式为例）

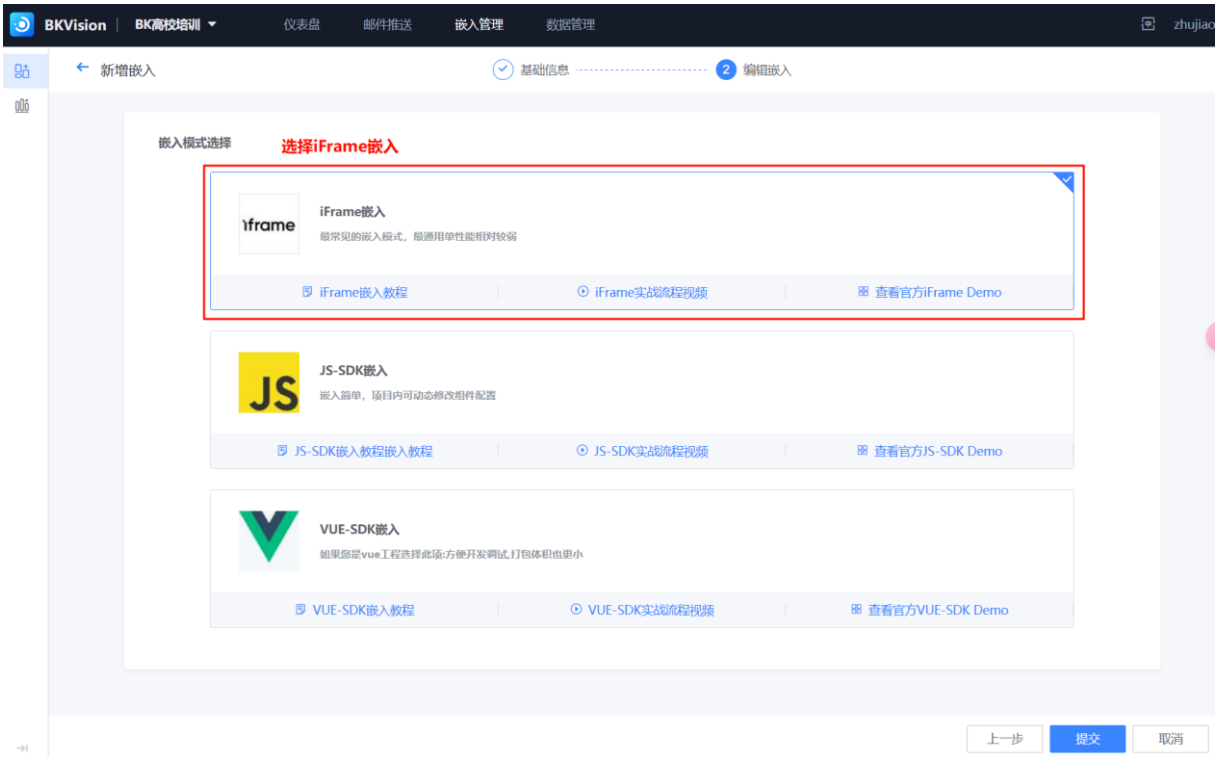
在线上的 BKVision 平台完成仪表盘的配置、编辑后，我们需要将仪表盘嵌入到我们自己的前端应用

1. 首先，在 BKVision 平台->嵌入管理(【BK 高校培训】空间下)->新增

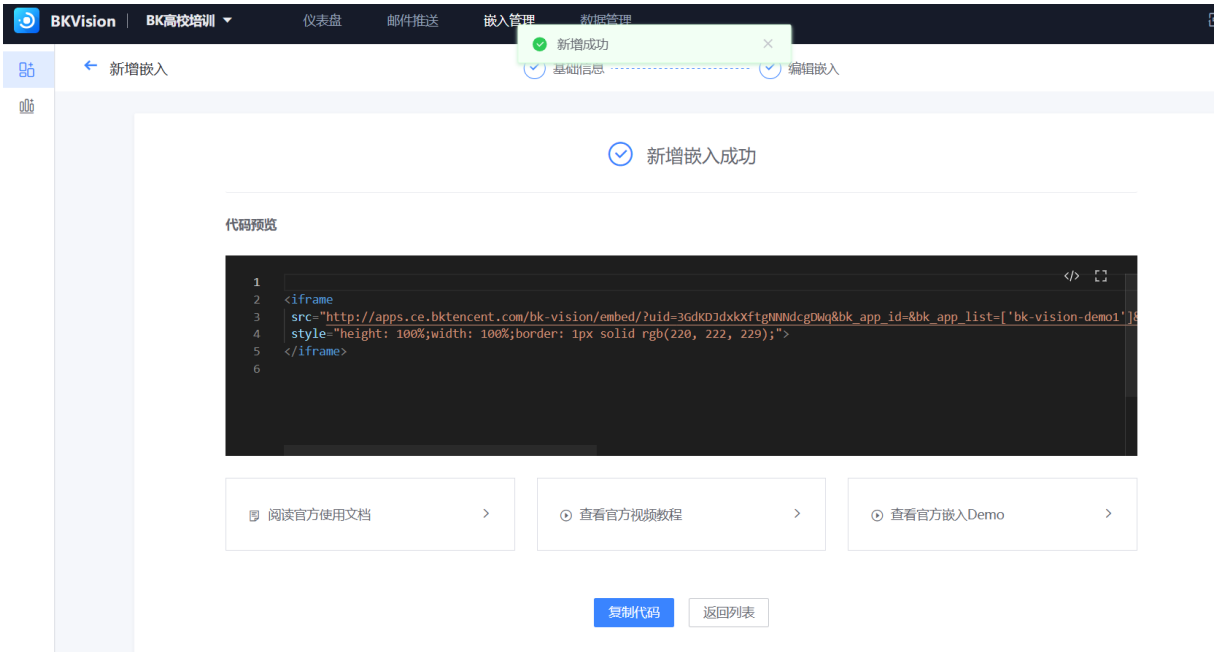
2. 选择需要嵌入的仪表盘及嵌入目标应用的应用 ID：



3. 选择嵌入方式，此处选择 iFrame 进行嵌入，这里的一些教程链接无法打开，嵌入代码比较简单，参考本实验手册即可

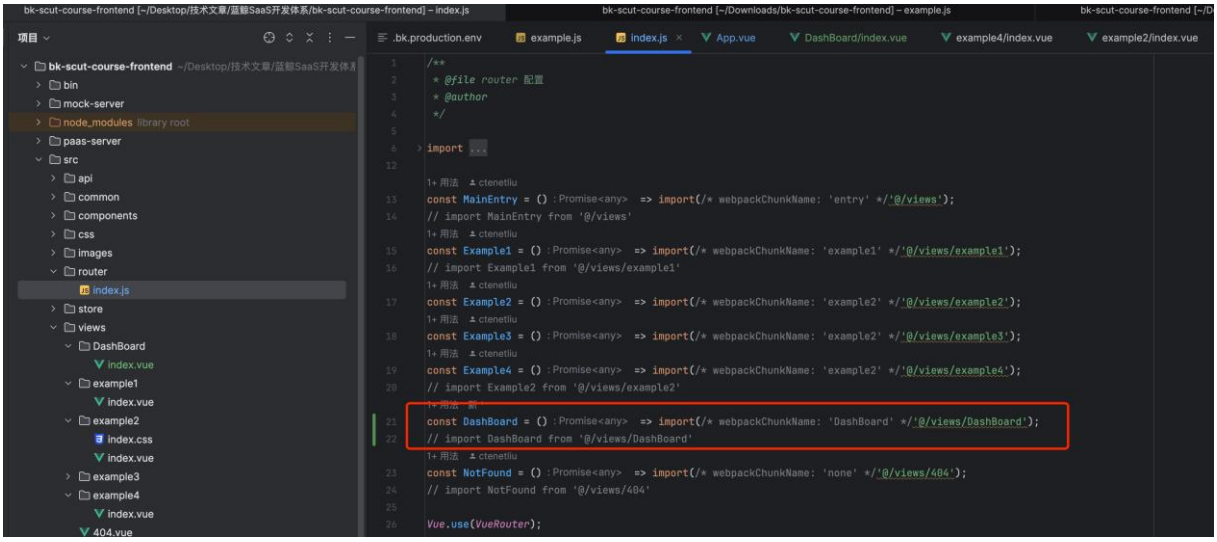


选择嵌入方式后，系统会自动生成一段 iFrame 代码，后续复制到自己的前端应用中：

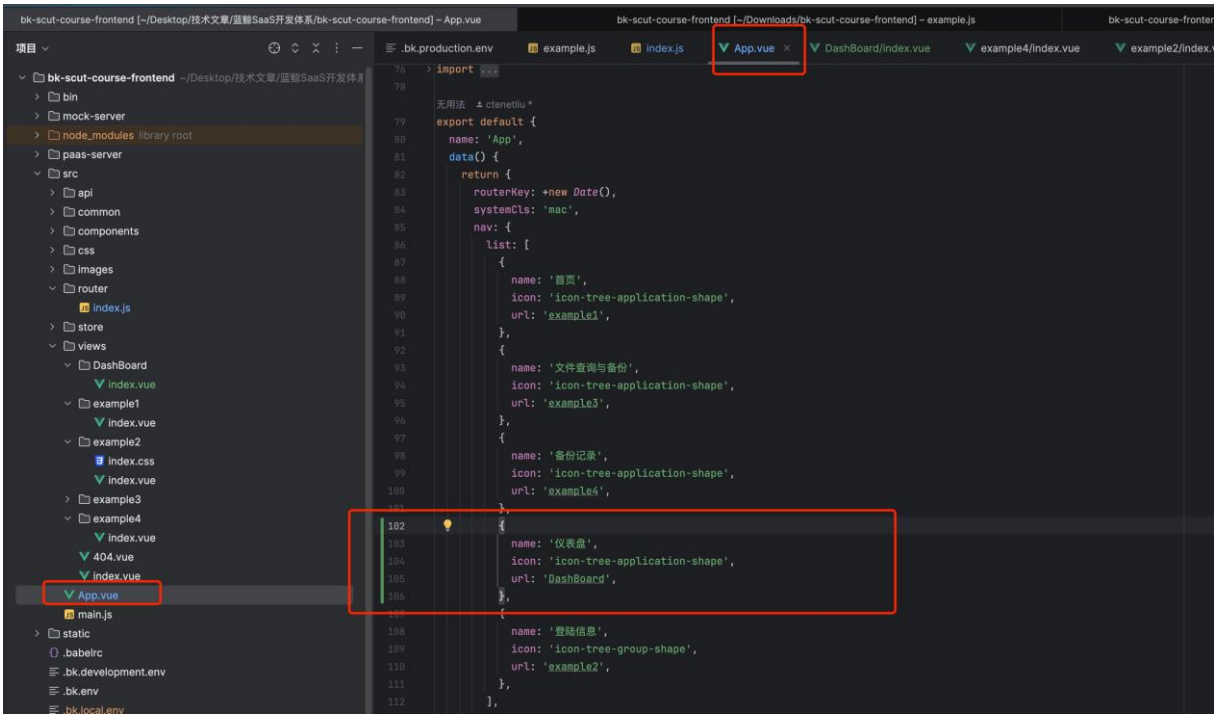


4. 在自己的前端应用中新增页面：

   a. 在前端代码的 router/index.js 中新增 DashBoard 页面的配置



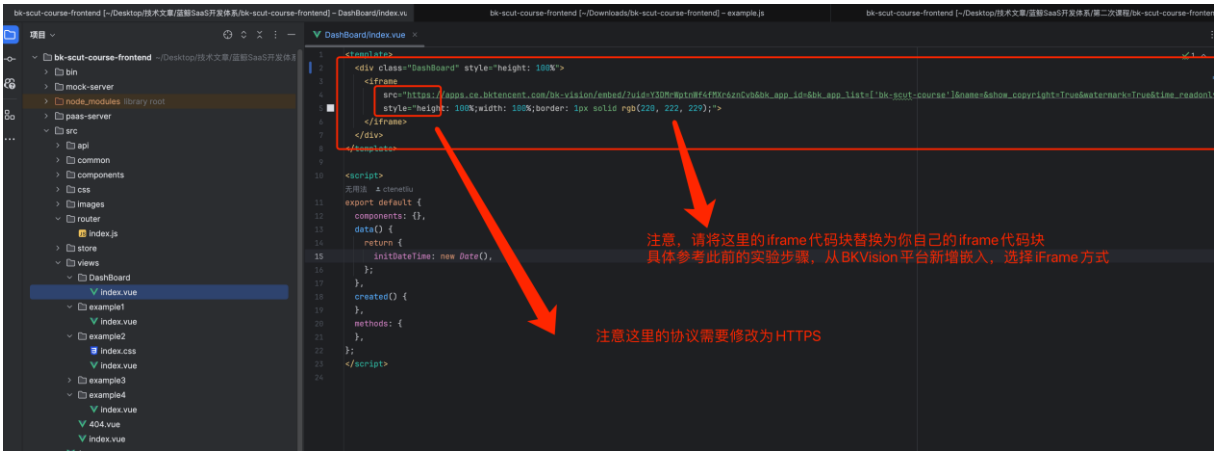
b. 在 src/App.vue 中新增 DashBoard 相关配置



c. 在 src/views 目录下新增文件夹 DashBoard，在 src/views/DashBoard 目录下新增 Vue 组件文件 index.vue，代码如下

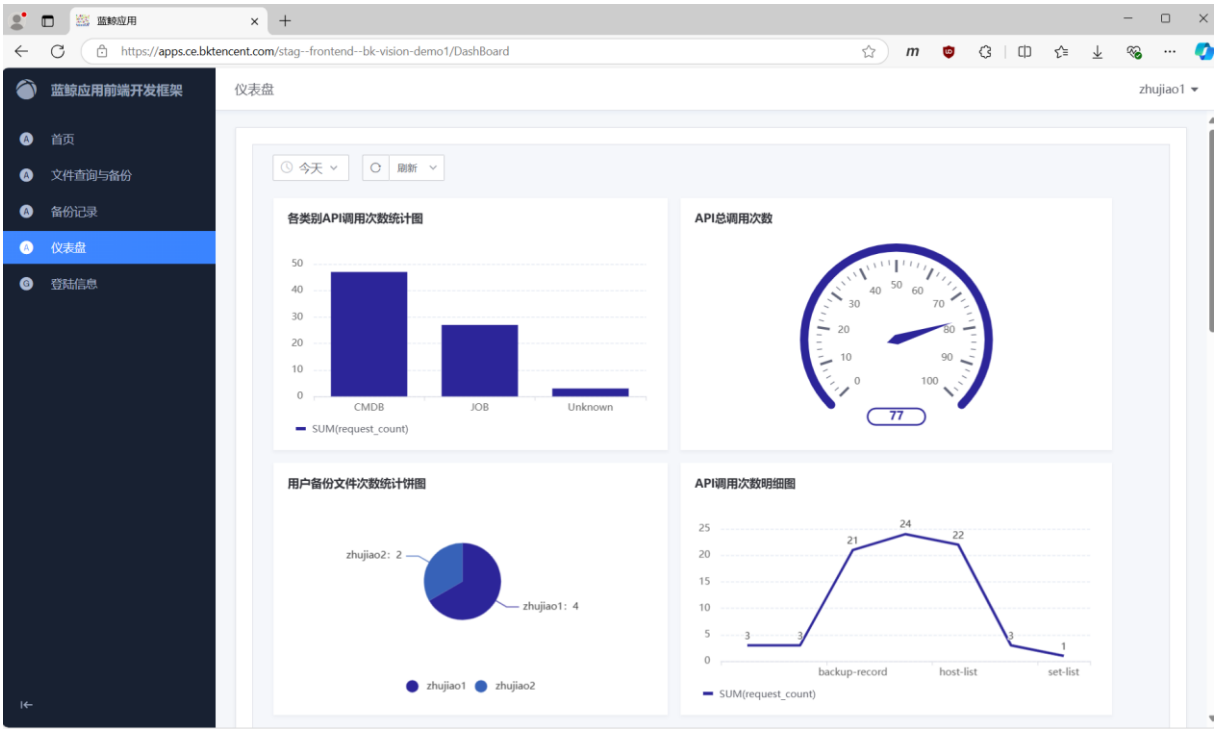
注意，BKVision 平台自动生成的 iFrame 代码默认使用 HTTP 协议，这里我们需要修改为 HTTPS 才能够让我们的应用正常访问

```
<template>
  <div class="DashBoard" style="height: 100%">

  </div>
</template>

<script>
export default {
  components: {},
  data() {
    return {
      initDateTime: new Date(),
    };
  },
  created() {
  },
  methods: {
  },
};
</script>
```



5. 发布仪表盘并推送代码到远程仓库，部署上线

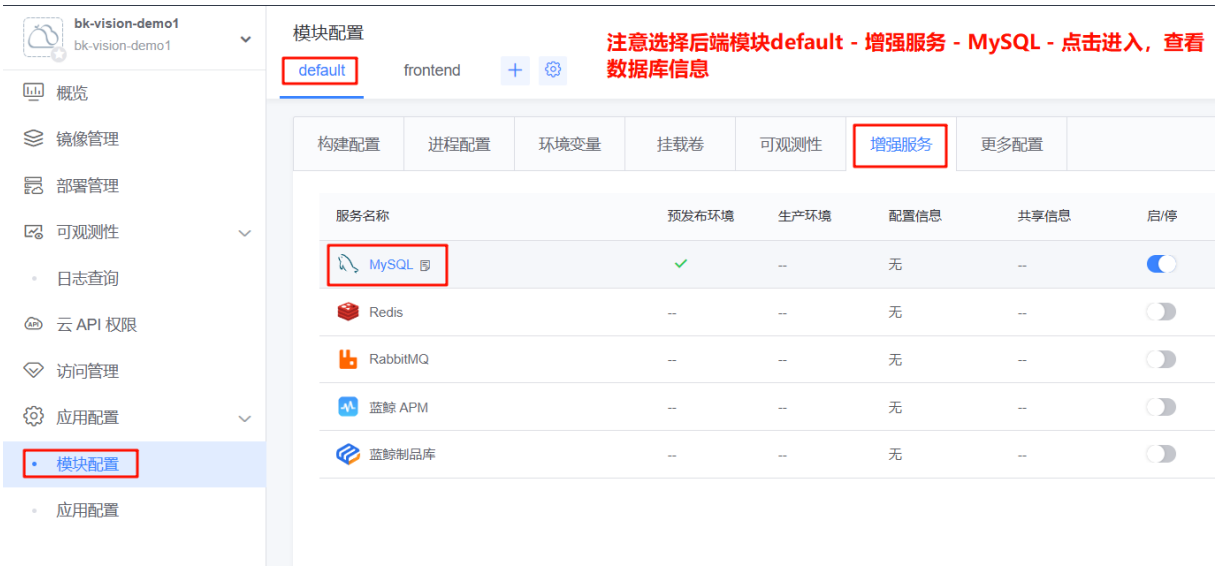
6. 查看效果



QA 环节

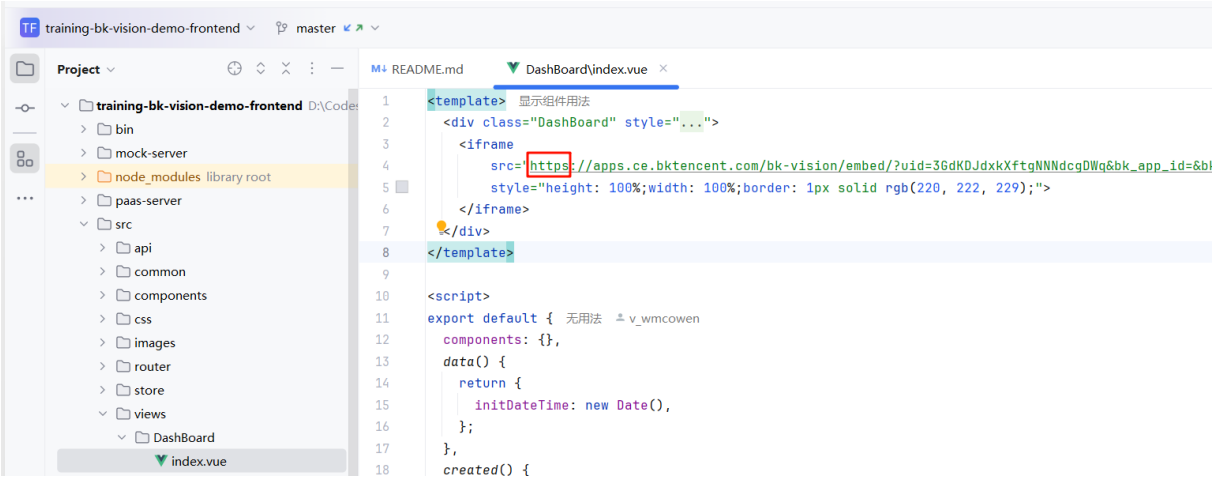
Q1、数据源配置信息从哪里获取？

从开发者中心，获取自己的 SaaS 应用的数据库连接信息



Q2、为什么我新增仪表盘后，无法访问？

请确认你的仪表盘已经开启，另外，请确保你选择了正确的应用 ID，协议使用 HTTPS：



此外，本地无法访问到数据，请将你的代码推送到远程仓库并部署上线后再次访问